

RelPT 是什么，为什么这次实验要做它

RelPT 不是新的 PPO loss，也不是新的大模型。它是 post-training 前处理阶段的关系型控制层：把 prompt、trajectory、reward、logprob、advantage、batch 等中间产物放进 append-only tables，用 SQL/join/cache 判断哪些东西已经算过，哪些东西还缺，然后只调用缺失部分的昂贵 executor。

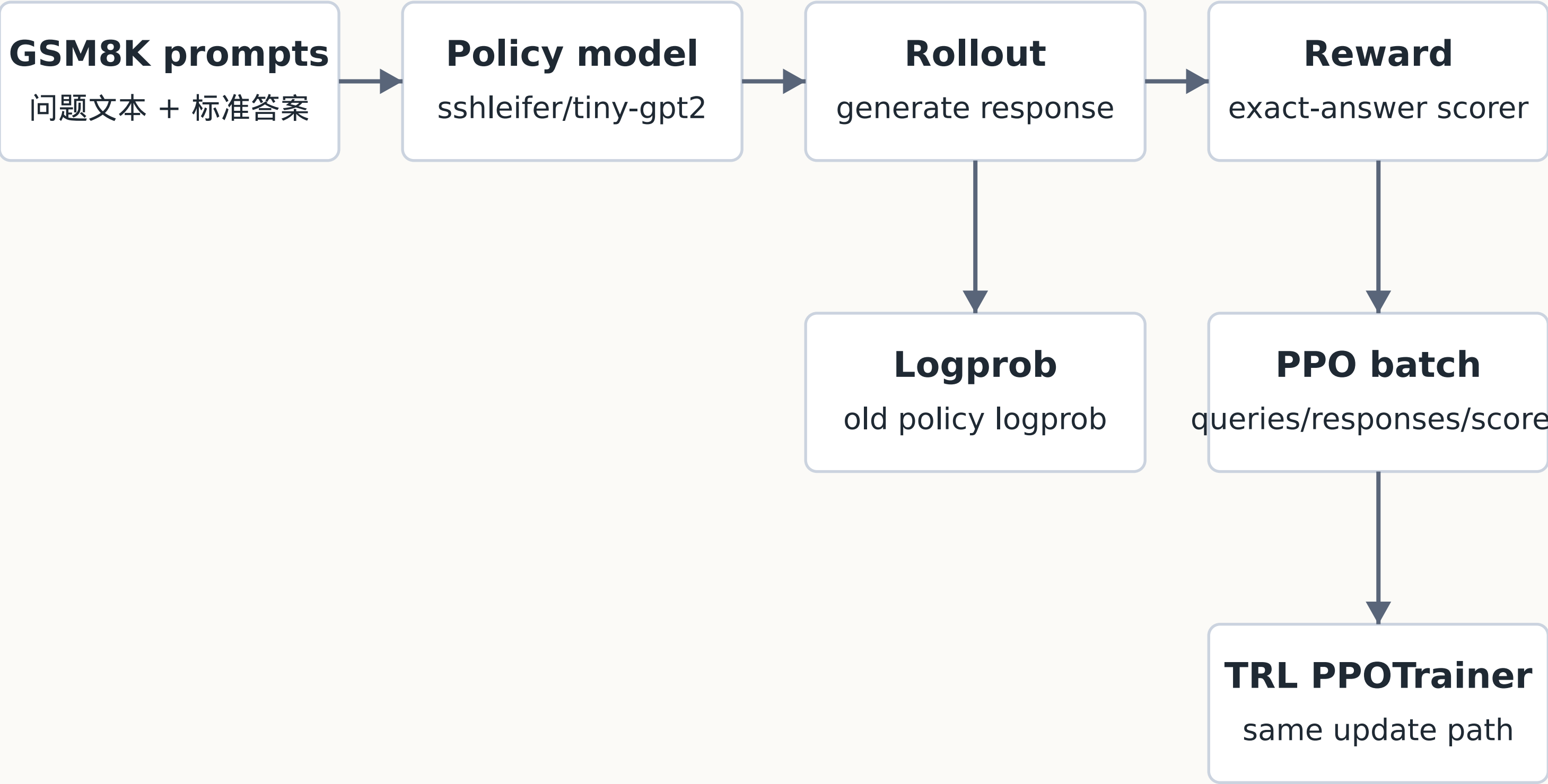
一句话版本

Baseline 每次重新准备 PPO batch；RelPT 先查表，能复用 rollout/reward/logprob/batch 的就复用，只补缺口。PPO trainer 本身保持不变。

本报告覆盖

1. PPO baseline 从 prompt 到 update 的完整流程
2. RelPT 加在哪一层，和 PPO/TRL/vLLM/reward model 的边界
3. RelPT 使用哪些表、哪些 join、什么 SQL optimizer
4. 当前仓库中两套实现：完整 SQLite 原型 + TRL PPO cache 版本
5. 最新 200-step tiny-gpt2/GSM8K 结果图，以及结果应该如何解读

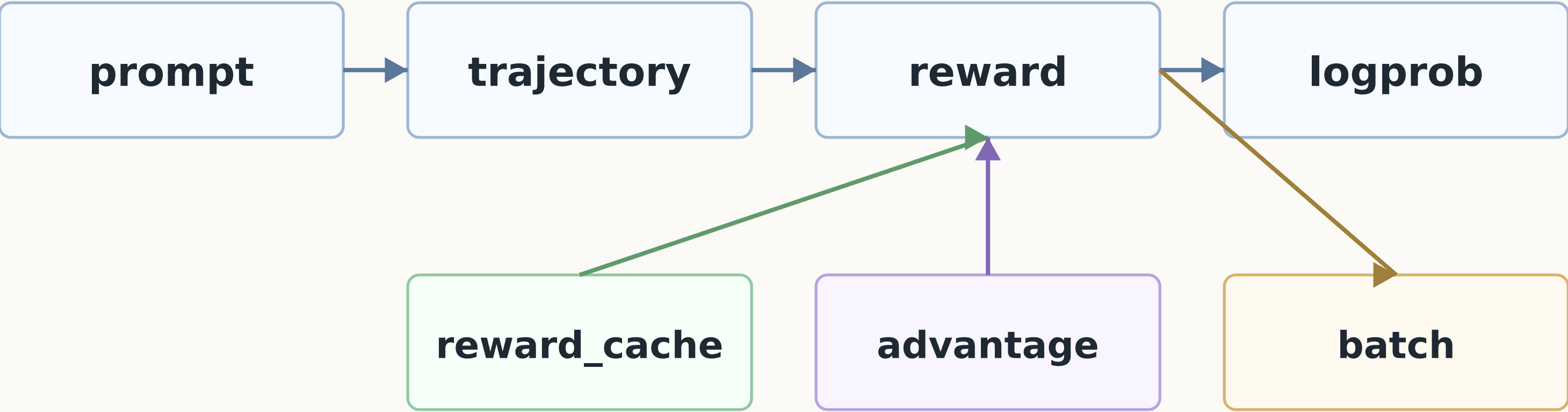
没有 RelPT 时 : PPO batch preparation 每次重跑



baseline 的问题

如果同一个 policy step 的同一批 prompt 因为 retry、失败恢复、调参、K 从 4 增加到 6、或 reward/logprob 补算而再次准备 batch , baseline 往往重新 generate、重新 reward、重新 logprob。对于真实 LLM , generate 和 reward model 调用很贵。

加上 RelPT 后：先查关系表，只补缺失行



RelPT optimizer 做的事情

- 1

LEFT JOIN prompt/trajectory
找当前 policy 下缺多少 rollout
- 2

JOIN trajectory/reward_cache
同一 prompt + 同一 response hash 的 reward 直接复用
- 3

LEFT JOIN trajectory/logprob
只对缺 logprob 的 trajectory 调 executor
- 4

JOIN 完整 rows
materialize 成 PPO batch 交给 trainer

RelPT 的表和关系：不是 magic , 是 SQL materializati

prompt	输入 prompt、GSM8K 问题、标签/metadata
policy	PPO 更新前后的 policy version ; 每次 update append 一个版本
trajectory	rollout 结果 : prompt_id、policy_id、response、response_hash
reward	某个 reward_model 对 trajectory 的 reward
reward_cache	按 reward_model_id + prompt_id + response_hash 复用确定性 reward
logprob	PPO 需要的 old-policy logprob
advantage	return / advantage , 中间训练量
batch	materialized PPO batch membership
update_record	PPO 输入 policy、输出 policy、batch、trainer backend lineage
metric	训练和系统指标

SQL optimizer 是什么？

当前实现用 Python 标准库 sqlite3。SQLite 负责真实 SQL 执行、索引/unique constraint 和本地 query planning。RelPT 自己的 optimizer 不是替代 SQLite , 而是在 prepare_ppo_batch 前生成 worklists : rollout_requests、reward_traj_ids、logprob_traj_ids、advantage_traj_ids。然后 SQLite join 把已有 rows 和缺失 rows 做分因为 post-training 的昂贵部分通常在表外 executor , 例如 LLM generation、reward model、logprob engine。SQL 层的任务是避免重复调用这些 executor。

这个仓库里的具体实现

完整 RelPT SQLite 原型

文件：relpt.py。核心类 RelPT 创建 append-only SQLite tables。RelPT.plan_ppo_batch 生成缺失 worklist；RelPT.prepare_ppo_batch 调用 rollout/reward/logprob/trainer adapters。测试 test_relpt.py 验证同一个 PPO batch retry 不再调用昂贵 executor。

GSM8K simulated benchmark

文件：relpt_gsm8k_ppo.py。prompt 来自本地 GSM8K Arrow cache；rollout/PPO executor 是 toy softmax policy 的模拟器。它验证完整 RelPT 表结构能省 rollout/reward/logprob 行数，但不是实模型质量实验。

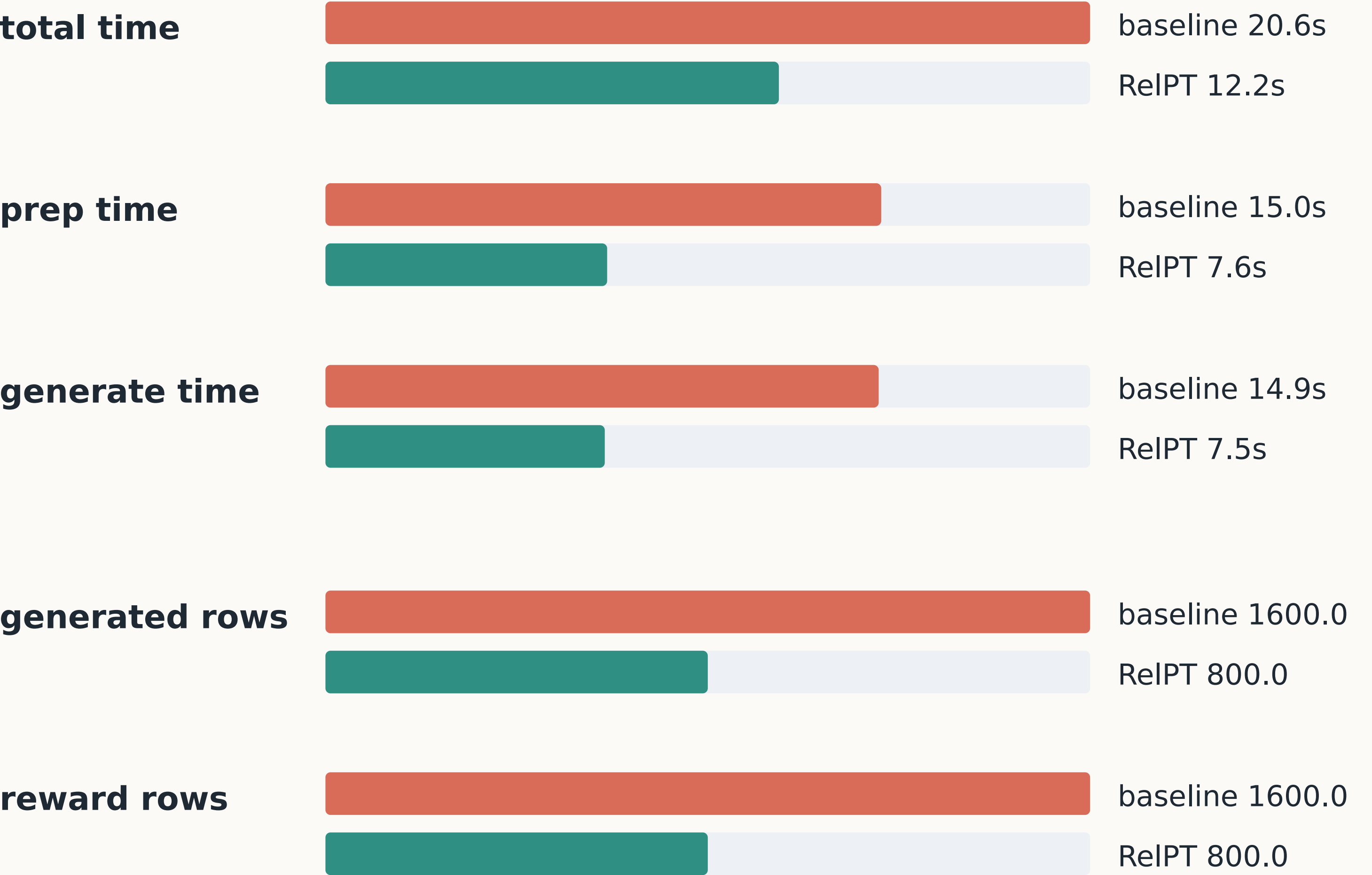
真实 TRL PPO same-framework 对照

文件：trl_gsm8k_relpt_ppo.py。baseline 和 RelPT 都调用同一个 TRL PPOTrainer，同一个模型 sshleifer/tiny-gpt2，同一个 GSM8K local Arrow cache。这里的 RelPT 是较小的 prepared-batch cache，目的是证明不改变 PPO update path 时也能减少重复 batch preparation。

最新 200-step TRL PPO : ReIPT 前后差异

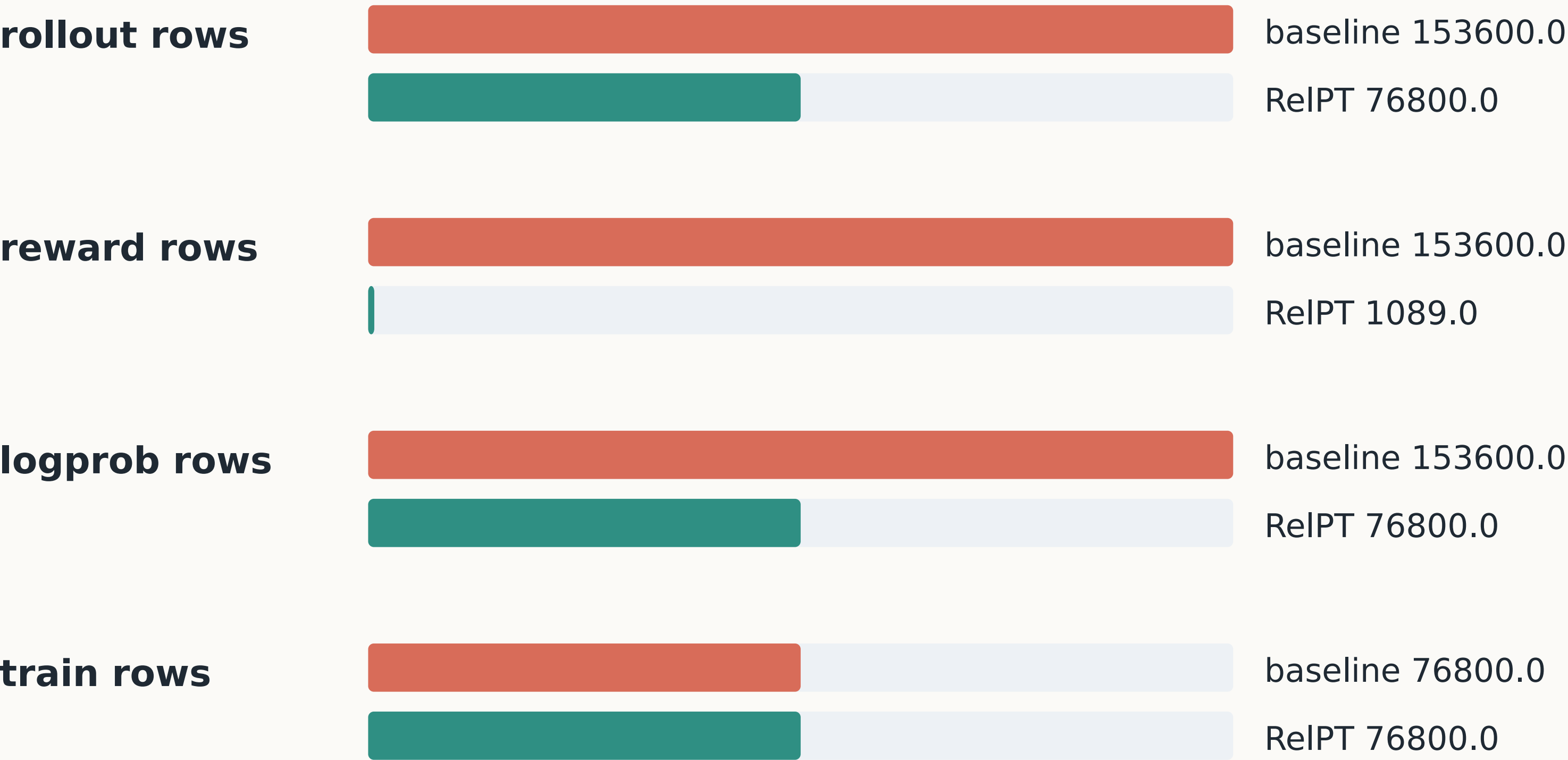
实验配置

```
model = sshleifer/tiny-gpt2
dataset = pyarrow_arrow_cache
framework = trl_ppo_0.11.4
steps = 200, batch_size = 4, max_new_tokens = 16
job output = runs/metrics/trl_ppo_relpt_17266929.json
```



总时间节省 40.7% ; 生成行数和 reward 行数各减少 50.0%。
final_eval_reward_delta = 0.0 , 说明这次实验主要证明系统省计算 ; 它不证明 tiny-gpt2 数学能力提升。

完整 RelPT 表结构的 200-step 模拟结果



这张图的含义

在完整 SQLite RelPT 原型里，rollout_rows 省 50.0%，logprob_rows 省 50.0%，reward_rows 省 99.3%。train_rows 省 0.0%，这是故意的：RelPT 不改 PPO 更新本身，只优化 PPO 前后的数据准备和复用。

reward_rows 省得最多，是因为 reward_cache 可以跨 policy version 复用 deterministic exact-answer reward。

从开始到结束：应该怎样理解 RelPT

PPO on what model?	最新真实 TRL 对照使用 sshleifer/tiny-gpt2，在本地 GSM8K Arrow cache 上跑 200 steps。
加 RelPT 前	每次 batch preparation 都重新 generate/reward/logprob；retry 或重复准备会浪费昂贵 executor 调用。
加 RelPT 后	先查关系表/cache。已有 trajectory/reward/logprob/batch 直接 materialize；缺什么补什么。
SQL optimizer	SQLite 是底层 SQL engine。RelPT 的 delta planner 负责生成缺失 worklist，SQLite join/constraint 负责复用和去重。
为什么这么做	真实 post-training 的成本通常不在 SQL，而在 LLM generation、reward model 和 logprob engine。把中间产物表化后，就能避免重复调用这些黑盒 executor。
结果怎么读	最新 TRL 200-step 总时间省 40.7%，generation/reward rows 各省 50%。质量 delta 为 0，说明 PPO 更新路径没变；这次主要是系统效率证据。

最短结论：RelPT 是 post-training pipeline 的关系型控制层。它不声称 PPO loss 更好，而是让同一个 PPO trainer 少重复准备数据。